TECHWIZ 7 · OFFICIAL INTERVIEW COMPONENT · 3-MEMBER TEAM DEFENSE GUIDE

SkillSprint AI — 3-Member Team Interview Guide

Lead/Backend & AI Developer · Frontend Developer · Documentation Specialist

Aptech TECHWIZ 7 · Generative AI PowerPlay · OnboardVerse
Simple English Words, Short Answers (30–45 Seconds per Question)

---

1. Project Overview & Main Objectives

Simple Answer:

"Our project is **SkillSprint AI**, an employee onboarding and training system for **Nexora Labs**.

Our main goal is to create personalized 90-day learning plans for new employees so they can start their work quickly and follow all company rules.

We use a simple **two-step system**:

- **Step 1 (AI):** AI writes the personalized training plan with tasks, checklists, and quizzes.
- **Step 2 (Python):** Pure Python code checks the plan against real company policies to make sure the AI did not make up any fake rules or miss anything mandatory."

2. Project Features and Functionality

Simple Answer:

"SkillSprint AI has five main features:

- **Document Upload:** Admins upload company policy files (PDF and Word). The system checks file types, rejects fake files, and prevents duplicate uploads.
- **Rule Matrix:** Python reads all company rules and organizes them by job role (like Backend Developer or QA Engineer).
- **AI Plan Maker:** Generates Day 1 to Day 90 plans with learning modules, checklists, and quizzes.
- **Python Checker (13 Rules):** Verifies that all required rules are covered, citations are real, and no contradictory rules exist.
- **Smart Policy Updates:** When a company policy changes, the system updates only the affected modules without deleting the employee's completed quiz scores."

3. Technologies and Tools Used

Simple Answer:

"We used modern and reliable tools:

- **Backend:** Python and FastAPI to build the fast web server.
- **Database:** PostgreSQL with `pgvector` to store data and search text quickly.
- **AI Models:** Claude, Google Gemini, and local Ollama to generate learning plans.
- **Frontend:** React and TypeScript with Vite to create a clean, responsive web interface.
- **Testing & Tools:** Docker to run everything easily, and Pytest with 232 automated tests."

4. Your Individual Contribution (3-Person Team Breakdown)

Member 1: Team Lead & Lead Developer (Backend & AI)

"I am the Team Lead and lead developer. I built the backend server using FastAPI and set up the PostgreSQL database with vector search. I also built the AI generation pipeline (Claude/Gemini/Ollama) and wrote the 13 Python validation rules that check the AI plans with zero AI imports."

Member 2: Frontend Developer & UI Integration

"I built the web user interface using React and TypeScript across 18 pages. I created the login page, the plan viewer, and the employee quiz page, and connected the frontend with all backend APIs so users can complete tasks and see live progress."

Member 3: Documentation & System Analyst

"I managed the complete technical documentation for the project. I authored the comprehensive SRS report, designed the 6 system architecture and UML diagrams, wrote the testing reports and user manual, and ensured all project deliverables matched the software 100%."

5. How Your Team Worked Together

Simple Answer:

"Our 3-member team worked closely using simple agile practices:

- **Daily 15-Minute Meetings:** Every morning we checked what was done, what was next, and solved any blockers.
- **Clear Team Roles:** Member 1 focused on the Backend & AI, Member 2 focused on the Frontend UI, and Member 3 managed the complete Documentation and system diagrams.
- **Data Planning First:** We agreed on API formats first so the frontend and backend could code smoothly while documentation was prepared in parallel.
- **Always Testing:** We made sure our 232 automated tests always passed before finishing any feature."

6. How Tasks Were Divided Among Team Members

Simple Answer:

"We divided the project clearly into three distinct areas:

- **Member 1 (Lead / Backend & AI):** Backend API, PostgreSQL database, AI generation prompts, and pure Python validation rules.
- **Member 2 (Frontend & UI):** React 19 single-page app, 18 web views, user authentication, and API integration.
- **Member 3 (Documentation & Analysis):** SRS documentation, UML/architecture diagrams, user guides, testing reports, and presentation slides.

We held weekly walkthroughs so all three members completely understand the entire system."

7. Challenges Faced During the Project

Simple Answer:

"We faced four main practical challenges:

- **AI Making Up Fake Rules:** AI models sometimes invent fake policy numbers or skip required tasks.
- **File Size & Slow Responses:** Sending 39 full documents to the AI was too large and very slow.
- **Trick Text in Files:** Some test documents had hidden text trying to trick the AI into approving everything.
- **Keeping Docs in Sync:** As developers added new features quickly, documentation had to be updated rapidly to match the code."

8. How Your Team Solved Those Challenges

Simple Answer:

"We solved them simply and cleanly:

- **Pure Python Checking:** We used 13 strict Python rules to check the AI's plan against real database records.
- **Backup AI Models:** If one AI fails or is slow, the system automatically switches to Gemini, Ollama, or an offline mode.
- **Security Scanner:** We scan uploaded text first and remove any suspicious prompt text before calling the AI.
- **Daily Sync for Documentation:** Developers gave live demos of any code changes so the documentation member could update diagrams and reports immediately."

9. Difficulties or Conflicts Within the Team & How Handled

Simple Answer:

"We had a real team challenge regarding **feature changes versus documentation deadlines**:

As our two developers were quickly improving the API and adding new features, our documentation member was finding it hard to keep the SRS report and architecture diagrams up to date.

How We Handled It:

We set up a quick 10-minute daily demo. The developers walked through all new endpoints and UI screens, and provided sample API responses. This allowed our documentation specialist to update all diagrams and reports accurately without any delay or confusion."

10. What You Learned from Working as a Team

Simple Answer:

"We learned three important lessons:

- **Good Communication:** Daily sync-ups between developers and the documentation writer kept everyone on the same page.
- **AI Needs Traditional Code:** In real enterprise software, AI cannot be trusted blindly; pure Python rules are necessary to verify results.
- **Team Support:** Having clear ownership while helping each other ensured all 3 members finished the project on time."

11. What Would You Improve If You Had More Time

Simple Answer:

"If we had more time, we would add:

- **Voice AI Practice:** A voice assistant so employees can practice speaking to customers and managers.
- **In-Browser Coding Practice:** Safe coding boxes in the browser for software trainees to write real code.
- **HR System Sync:** Automatically connect to tools like Workday to import new employees.
- **Short Quiz Reminders:** Send quick 1-minute refresher quizzes on Slack or Teams."

Rapid-Fire Examiner Grilling Cheat Sheet (Simple 1-Line Answers)

Question	Simple 1-Sentence Answer
Why not just use LangChain?	*"Because LangChain relies only on AI; we added pure Python rules to guarantee 100% accurate compliance."*
Why pgvector over Pinecone?	*"PostgreSQL keeps our normal tables and search data together in one place, so it is faster, simpler, and free."*
What if the internet goes down?	*"The system automatically runs our local Ollama model or offline engine, so it never stops working."*
When is a plan considered 'Verified'?	*"When 100% of required rules are covered, all citations are real, and there are zero policy conflicts."*
Why is there no AI in Pipeline 2?	*"Because we want our validation to be 100% consistent and rule-based every single time."*
How do you stop SQL injection?	*"We use SQLAlchemy safe queries where user input is never executed as raw database code."*
How fast is plan generation?	*"A full 30-module plan generates and gets checked by Python in under 20 seconds."*
What happens if an employee fails quizzes?	*"The system marks that topic as a Weak Area and suggests review lessons on their dashboard."*